Two-Tower Matrix Multiplication: A Depth-Efficient Quantum Subroutine for Matrix Chain Products

Giacomo Antonioli
Department of Computer Science, University of Pisa, Italy

Anna Bernasconi
Department of Computer Science, University of Pisa, Italy

Alessandro Berti[1]
Department of Computer Science and Department of Physics, University of Pisa, Italy

Gianna M. Del Corso
Department of Computer Science, University of Pisa, Italy

Alessandro Poggiali
Department of Computer Science, University of Pisa, Italy

Abstract

Matrix chain multiplication — computing $\mathcal{W} = M^{(0)} \cdots M^{(K-1)}$ where $M^{(k)} \in \mathbb{R}^{P_k \times P_{k+1}}$ — arises in scientific computing, machine learning, and graph analysis. Despite the importance of this problem, for chains of distinct matrices, the classical number of operations grows linearly with the chain length K and polynomially in the matrix dimensions. We present *Two-Tower Matrix Multiplication*, a quantum subroutine that encodes the product $\mathcal{W}$ of the K matrices into a quantum state in circuit depth $\mathcal{O}(\max_k \operatorname{polylog}(P_k P_{k+1}))$, which is *independent of* K, whereas the qubit count is $\mathcal{O}\left(\sum_k \log P_k\right)$. The construction interleaves state-preparation operators across two layers; within each layer, all operators act on disjoint registers and execute in parallel. This subroutine can be specialized for the chain-vector case, which computes the product of $K-1$ matrices applied to a vector. We prove the correctness of the subroutine for all K and provide two implementations using the Qiskit and QCLAB frameworks. The subroutine is applicable to any downstream quantum algorithm that operates on a matrix encoded in the statevector, including norm estimation, graph-matrix powers, linear system solving, and quantum machine learning kernels.

2012 ACM Subject Classification Theory of computation → Quantum computation theory; Theory of computation → Design and analysis of algorithms; Mathematics of computing → Linear algebra

Keywords and phrases quantum algorithms, matrix multiplication, state preparation, quantum subroutines, circuit depth, quantum linear algebra

Acknowledgements This study was carried out within the National Centre on HPC, Big Data and Quantum Computing - SPOKE 10 (Quantum Computing) and received funding from the European Union Next-GenerationEU - National Recovery and Resilience Plan (NRRP) – MISSION 4 COMPONENT 2, INVESTMENT N. 1.4 – CUP N. I53C22000690001. Partial support was also provided by the INdAM - GNCS project "Algebra Lineare Quantistica, State Preparation e Compilazione di Circuiti Quantistici", CUP E53C25002010001, and by the Italian Project Fondo Italiano per la Scienza FIS00001966 "MIMOSA". G. Del Corso is also partially supported by European Union - NextGenerationEU under the National Recovery and Resilience Plan (PNRR) - Mission 4 Education and research - Component 2 From research to business - Investment 1.1 Notice Prin 2022 - DD N. 104 2/2/2022, titled Low-rank Structures and Numerical Methods in Matrix and Tensor Computations and their Application, proposal code 20227PCCKZ – CUP I53D23002280006.J53D23003620006 and by the U.S. Department of Energy, Office of Science,

[1] Corresponding author

34th Annual European Symposium on Algorithms (ESA 2026).
Editor: TODO Editor; Article No. TODO; pp. TODO:1–TODO:17
Leibniz International Proceedings in Informatics
Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

TODO:2Two-Tower Matrix Multiplication

Table 1 Comparison of quantum subroutines for matrix chain multiplication. All matrices are assumed square of size N; K denotes the chain length. The column Encoding model specifies how the result is represented: "quantum state" means that the entries of the product appear as amplitudes of part of the final statevector; "block-encoded" means that the product is embedded as the top-left block of a unitary operator. For a fair comparison, all methods assume that the input matrices can be prepared/queried via an efficient state-preparation subroutine in time polylog(N); consequently, the costs reported may differ from those in the original papers.

Reference	Setting	Depth/Query	Qubits	Encoding Model
Li et al. [22]	$A^K b$	$\Theta(\sqrt{K}\,\mathrm{polylog}(N))$	$\mathcal{O}(\log N)$	block / quantum state
Montanaro & Shao [23]	A^K	$\mathcal{O}(\sqrt{K}\,\mathrm{polylog}(N))$	n/a	block-encoded
Fang et al. [12]	any K-tuple	$\mathcal{O}(K\,\mathrm{polylog}(N))$	$\mathcal{O}(\log K)$	block-encoded
This work	any K-tuple	$\boldsymbol{\mathcal{O}(\mathrm{polylog}(N))}$	$\boldsymbol{\Theta(K \log N)}$	**quantum state**

National Quantum Information Science Research Centers, Superconducting Quantum Materials and Systems Center (SQMS) under Contract No. 89243024CSC000002.

1 Introduction

Matrix chain multiplication — computing $\mathcal{W} = M^{(0)} \cdots M^{(K-1)}$ for K matrices — arises in different contexts, such as iterative solvers [15], deep feature maps [6], and graph spectral analysis [18, 24]. Classically, each $N \times N$ product costs $O(N^\omega)$ ($\omega < 2.372$, due to a sequence of improvements [7, 1, 11]), so a chain of K distinct matrices requires $O(KN^\omega)$ operations. Even with unbounded parallelism (tree-structured multiplication), the depth grows as $\mathcal{O}(\log K \cdot \log N)$ [15]. Quantum linear algebra subroutines [16, 28, 6] typically promise speedups provided that an efficient state-preparation subroutine is available to encode the data into quantum states. Two encoding models are relevant: *statevector encoding*, where matrix entries appear as amplitudes, and *block-encoding*, where the matrix is the top-left block of a unitary. Block-encoding approaches typically have a depth that grows with K [10]. In a straightforward implementation, the number of ancilla qubits grows at least linearly with K because each block-encoded matrix requires its own ancilla register. The overall subnormalization factor is $\prod_{k=0}^{K-1} \alpha_k$, where α_k is the subnormalization of the block encoding of $M^{(k)}$ and $\alpha_k \geq \|M^{(k)}\|_2$. When this factor is large, the success probability decreases, and the computation becomes more sensitive to errors. To reduce the ancilla overhead we can apply the *compression gadget* [12], which lowers the qubit count to $\max_k a_k + \log(K) + 1$, where a_k is the number of ancillae required to block-encode $M^{(k)}$, at the expense of increased circuit depth. Assuming efficient state-preparation, each block encoding has depth $\mathcal{O}(\mathrm{polylog}(N))$, the K sequential applications yield a total circuit depth of $\mathcal{O}(K\,\mathrm{polylog}(N))$; the overall subnormalization factor however, remains unchanged. Li et al. [22] address the special case of single repeated matrix A applied to a vector b, i.e., computing $A^K b$, in the block-encoding model achieving $\Theta(\sqrt{K})$ queries via Chebyshev approximation and QSVT [13]; no analogous speedup is known for chains of *distinct* matrices via this approach. Montanaro and Shao [23] show that for $f(x) = x^K$, the quantum query complexity of approximating an entry $\langle i| A^K |j\rangle$ of an s-sparse Hermitian matrix is $\mathcal{O}(\sqrt{K})$. This yields an exponential separation from the classical lower bound $\widetilde{\Omega}\Big((s/2)^{(\sqrt{k}-1)/6}\Big)$, and implying optimality of QSVT-based algorithms for matrix powers. In contrast, our setting is more general: the chain product $\mathcal{W}$ consists of matrices differing in both values and dimensions, and the QSVT framework does not directly apply. Crucially, none of the approaches above achieves circuit depth independent of K for chains of distinct matrices.

Table 2 Classical vs. quantum cost for the chain product of K distinct square $N \times N$ matrices ($\omega < 2.372$). The classical column counts arithmetic operations; the Two-Tower column counts circuit depth. The ordering cost is the cost of finding the optimal parenthesisation of the product. The qubit count refers to circuit registers only.

	Classical	**Two-Tower**
Ordering cost	$O(K^3)$ arith. ops	not needed
Execution (seq. depth)	$O(K\,N^\omega)$	$\mathcal{O}(\mathrm{polylog}(N))$
Memory / qubits	$O(K\,N^2)$ words	$\Theta(K \log N)$ qubits

In this work, we present the Two-Tower Matrix Multiplication, a quantum subroutine that generalises the two-matrix product of [4] to chains of arbitrary length K. Given matrices $M^{(k)} \in \mathbb{R}^{P_k \times P_{k+1}}$ for $0 \le k < K$, the subroutine encodes the chain product as amplitudes in circuit depth $\mathcal{O}(\max_k \mathrm{polylog}(P_k P_{k+1}))$, independent of K, using $\mathcal{O}\big(\sum_k \log P_k\big)$ qubits. We emphasize that this polylogarithmic circuit depth is achieved only when each matrix $M^{(k)}$ can itself be loaded into the quantum state in polylogarithmic time — for instance, when the entries of $M^{(k)}$ are stored in a quantum-accessible data structure such as a QRAM [14], which supports the preparation of the corresponding quantum state in $\mathcal{O}(\mathrm{polylog}(P_k P_{k+1}))$ gates. Table 1 compares our method with the existing approaches. For a fair comparison, we assume that all algorithms rely on an efficient state preparation subroutine that prepares/queries the matrix entries in polylogarithmic time.

Our subroutine produces an output state whose amplitudes encode the entries of the chain product $\mathcal{W}$ on a specific subspace; the remaining amplitudes are spurious terms that do not contribute to the result. The signal weight $\Omega_{\mathcal{W}} = \frac{\|\mathcal{W}\|_F^2}{\prod_{k=0}^{K-1} \|M^{(k)}\|_F^2}$, $\Omega_{\mathcal{W}} \le 1$, measures the total squared amplitude carried by the useful components. For well-conditioned matrices $\Omega_{\mathcal{W}}$ decays with K, whereas matrices with peaked spectral structure retain $\Omega_{\mathcal{W}}$ close to unity; in either case, Amplitude Amplification boosts $\Omega_{\mathcal{W}}$ to any desired constant at an additional cost of $\mathcal{O}(1/\sqrt{\Omega_{\mathcal{W}}})$ repetitions. This quantity plays the same role as the subnormalization factor $\prod_k \alpha_k$ in block-encoding approaches — the two are directly comparable — and subnormalization affects all quantum linear algebra subroutines, not just the Two-Tower.

Two observations inspire the proposed subroutine. (1) Encoding the entries of a matrix as amplitudes of a quantum state is equivalent to mapping it to its row-wise vectorization. (2) Classical matrix theory expresses the vectorization of a three-factor product AXB through the Kronecker product: $\mathrm{vec}(AXB) = (B^T \otimes A)\,\mathrm{vec}(X)$ [17], and this identity extends recursively to longer chains. The Two-Tower subroutine draws from this decomposition: preparing the first and last matrices of the chain, $M^{(0)}$ and $M^{(K-1)}$, on separate registers realises their Kronecker product naturally within the quantum circuit, while a contraction mechanism pairs matching indices from adjacent matrices and accumulates their products through quantum superposition, yielding the inner sums that define the chain product. The final quantum state thus contains the vectorization of the full chain product. This construction applies to chains with an odd number of matrices; chains with an even number require a minor circuit-level adjustment inspired by the two-matrix subroutine of [4].

Amplitude-based quantum algorithms can leverage the Two-Tower output state directly [2, 3, 26, 27]. We observe that, as with other quantum routines such as the Quantum Fourier Transform [25], extracting all entries by tomography negates the advantage; the subroutine is beneficial when the downstream computation requires only aggregate quantities (norms, traces, inner products) extractable via Amplitude Estimation [16, 28].

Table 2 summarises the classical and quantum costs for a chain of K square $N \times N$

matrices; the general rectangular case follows by replacing N^2 with $P_k P_{k+1}$. The qubit count $\Theta(K \log N)$ grows linearly with K — an exponential compression over the classical memory $O(KN^2)$, but not K-free. In general, the Two-Tower trades circuit depth for qubit count.

The contributions of this work are:

- (1) a subroutine with circuit depth $\mathcal{O}(\max_k \operatorname{polylog}(P_k P_{k+1}))$, independent of K, improving over all prior methods whose depth depends on K;
- (2) a correctness proof for arbitrary K;
- (3) a chain-vector specialisation for the case $P_K = 1$;
- (4) open-source Qiskit [19] and QCLAB [20] implementations[2].

The remainder of this paper is organized as follows. Section 2 fixes notation, recalls the QRAM-based state-preparation model, and defines the matrix chain problem. Section 3 presents the Two-Tower circuit: Section 3.1 reviews the two-matrix base case and its rearrangement into the two-layer structure; Section 3.2 generalises to arbitrary K, formalises the register layout and the operators $\mathcal{OP}(L)$, $\mathcal{OP}(R)$, and provides an intuition for $K = 3$ walkthrough. Section 4 proves correctness for odd and even K, derives the depth and qubit bounds, and analyses the normalisation factor and signal weight. Section 5 specialises the subroutine to matrix-chain-vector products. Section 6 summarises the contributions and identifies open directions for future work.

2 Preliminaries

This section establishes notation, introduces the QRAM-based state-preparation model, reviews the two-matrix construction of [4] that the Two-Tower generalises, and defines the matrix chain multiplication problem. We recall the key concepts; for a full treatment see [25].

2.1 Notation

An *n-qubit quantum state* is a unit vector $|\psi\rangle \in \mathbb{C}^{2^n}$ and $|i\rangle^n$ denotes the computational-basis state encoding $i \in [0, 2^n)$, with the superscript omitted when clear from context. An *n-qubit unitary* (also called a *quantum gate* or *quantum operator*) is a matrix $U \in \mathbb{C}^{2^n \times 2^n}$ satisfying $UU^\dagger = \mathbb{I}_{2^n}$. We fix the notation $|\psi\rangle^{\text{size}}_{\text{name}}$ where the superscript denotes the number of qubits and the subscript identifies the register; both are omitted when clear from context.

For a matrix $A \in \mathbb{R}^{P \times R}$ with entries $a_{i,j}$, we write $\|A\|_F = \sqrt{\sum_{i,j} a_{i,j}^2}$ for its *Frobenius norm*, and $\|A_i\|$ for the Euclidean norm of the i-th row of A. We use lowercase letters for qubit counts: $p = \log P$, $r = \log R$, and more generally $p_k = \log P_k$, for a matrix chain. Throughout the paper, all matrices have real entries and all dimensions are assumed to be powers of two. We denote the conjugate transpose of a matrix or operator with the dagger symbol (†), and we refer to it as the adjoint of the matrix or operator. We now formalize the matrix chain multiplication problem.

▶ **Definition 1** (Matrix Chain Product). *Let $K \geq 2$. A* matrix chain *is a sequence of matrices $M^{(0)}, \ldots, M^{(K-1)}$ where each $M^{(k)} \in \mathbb{R}^{P_k \times P_{k+1}}$ with $P_k = 2^{p_k}$, for $k = 0, \ldots, K-1$. Denoting by $m^{(k)}_{i,j}$ the (i,j)-th entry of $M^{(k)}$, the* chain product $\mathcal{W} = M^{(0)} \cdots M^{(K-1)} \in$

[2] https://github.com/Brotherhood94/two_tower_quantum_matrix_multiplication_subroutine

$\mathbb{R}^{P_0 \times P_K}$ *has* (p, q)*-th entry* $w_{p,q}$ *given by*

$$w_{p,q} = \sum_{r_0,\ldots,r_{K-2}} \underbrace{m^{(0)}_{p,\,r_0}}_{\text{outer index } p} \underbrace{m^{(1)}_{r_0,\,r_1} m^{(2)}_{r_1,\,r_2} \cdots m^{(K-2)}_{r_{K-3},\,r_{K-2}}}_{\text{shared indices}} \underbrace{m^{(K-1)}_{r_{K-2},\,q}}_{\text{outer index } q},$$

where $p \in [0, P_0)$ *and* $q \in [0, P_K)$ *are the fixed outer indices, and each shared index* r_j *ranges over* $[0, P_{j+1})$.

The indices $r_0, \ldots, r_{K-2}$ are *shared indices*; they are summed over in the chain product. The outer indices $p \in [0, P_0)$ and $q \in [0, P_K)$ label the entries of $\mathcal{W}$.

2.2 State preparation

To build quantum algorithms for linear algebra, one must first efficiently load classical data — matrix entries — into quantum states. The dominant bottleneck is the *state-preparation step*: encoding an N-dimensional real vector into the amplitudes of a $\lceil \log N \rceil$-qubit state. Throughout this work, state preparation is assumed to operate under the *Quantum Random Access Memory* (QRAM) model of [5, 21]. Under this model, a quantum state encoding an N-dimensional real vector can be prepared in depth $\mathcal{O}(\text{polylog}(N))$ using $\Theta(\log N)$ qubits. Alternative data-loading models — such as block-encoding via sparse access [10] produce a different data representation (a block-encoding rather than the statevector encoding used here). All complexity bounds in this paper are stated within the QRAM model. With the QRAM model in place, we now define the state-preparation operator for a matrix.

▶ **Definition 2** (State-Preparation Operator). *Let* $M \in \mathbb{R}^{P \times R}$ *with* $P = 2^p$ *and* $R = 2^r$. *The* state-preparation operator $\mathcal{SP}(M)$ *is a unitary on* $p + r$ *qubits that maps*

$$\mathcal{SP}(M)\colon |0\rangle^p |0\rangle^r \longmapsto \frac{1}{\|M\|_F} \sum_{i=0}^{P-1} \sum_{j=0}^{R-1} m_{i,j} |i\rangle^p |j\rangle^r . \tag{1}$$

The operator decomposes as $\mathcal{SP}(M) = U(M)V(M)$, *where* $V(M)$ *prepares the row-weight state* $\frac{1}{\|M\|_F} \sum_{i=0}^{P-1} \|M_i\| \, |i\rangle$, *and* $U(M)$ *loads the normalised row* $\frac{M_i}{\|M_i\|}$ *[21]. Under the QRAM model,* $\mathcal{SP}(M)$ *can be realised in depth* $\mathcal{O}(\text{polylog}(PR))$ *using* $\Theta(\log(PR))$ *qubits. The adjoint satisfies* $\mathcal{SP}(M)^\dagger = V(M)^\dagger U(M)^\dagger$.

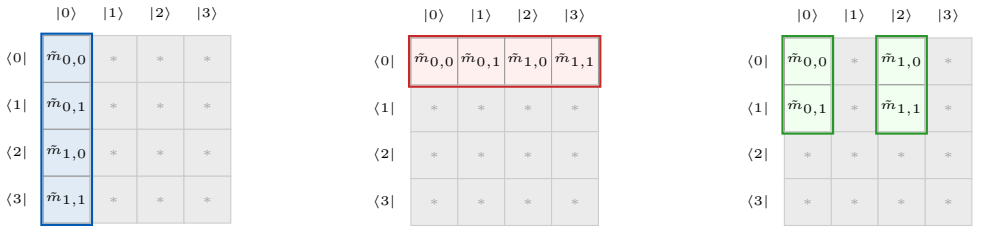
When the chain length K is even, the state-preparation operator of Definition 2 alone does not suffice to encode the last matrix. We therefore introduce the following operator.

▶ **Definition 3** (Modular-State-Preparation Operator). *Let* $M \in \mathbb{R}^{P \times R}$ *with* $P = 2^p$ *and* $R = 2^r$. *The* Modular state-preparation operator *is a unitary on* $p + r$ *qubits such that*

$$\mathcal{MSP}(M)\colon |i\rangle^p |0\rangle^r \longmapsto \frac{1}{\|M\|_F} \sum_{k=0}^{R-1} m_{i,k} |0\rangle^p |k\rangle^r + |\perp\rangle .$$

Given $\mathcal{SP}(M) = U(M)V(M)$ *the state-preparation operator defined in Definition 2, the Modular state-preparation operator can be decomposed as* $\mathcal{MSP}(M) = V(M)^\dagger U(M)$. *The term* $|\perp\rangle$ *denotes the superposition of all the other computational basis states, where the* p*-qubit register is not* $|0\rangle^p$.

Figure 1 illustrates the internal structure of the $\mathcal{SP}$, $\mathcal{SP}^\dagger$, and $\mathcal{MSP}$ unitaries for a concrete small matrix M. By Definition 2, the first column of $\mathcal{SP}(M)$ contains the vectorised, Frobenius-normalised entries of M; all remaining columns are fixed by unitarity



(a) $\mathcal{SP}(M)$ for $M \in \mathbb{R}^{2\times 2}$ where $\tilde{m}_{i,j} = \frac{m_{i,j}}{\|M\|_F}$.

(b) $\mathcal{SP}(M)^\dagger$ for $M \in \mathbb{R}^{2\times 2}$ where $\tilde{m}_{i,j} = \frac{m_{i,j}}{\|M\|_F}$.

(c) $\mathcal{MSP}(M)$ for $M \in \mathbb{R}^{2\times 2}$ where $\tilde{m}_{i,j} = \frac{m_{i,j}}{\|M\|_F}$.

Figure 1 Unitary matrices corresponding to **(a)** $\mathcal{SP}(M)$, **(b)** $\mathcal{SP}(M)^\dagger$, and **(c)** $\mathcal{MSP}(M)$ shown schematically for 2×2 matrix M. In $\mathcal{SP}(M)$, the first column (blue) contains the vectorised, Frobenius-normalised entries of M; the remaining columns (grey, marked $*$) are fixed by unitarity and play no role in the algorithm. In $\mathcal{SP}(M)^\dagger$, the first row (red) contains the normalised entries of M; the remaining rows are irrelevant. In $\mathcal{MSP}(M)$, the normalised entries (green) of M retain the row structure of the original matrix.

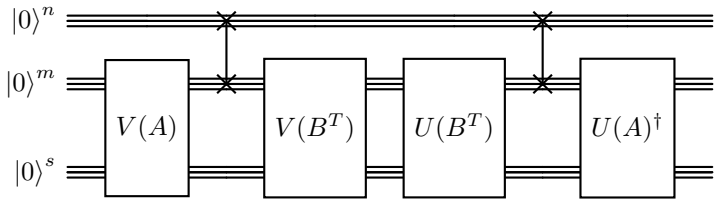


Figure 2 Original circuit for $C = AB$ from Bernasconi et al. [4], with $A \in \mathbb{R}^{M\times N}$, $B \in \mathbb{R}^{N\times S}$, $m = \log M$, $n = \log N$, $s = \log S$. The circuit uses four state-preparation operators ($V(A)$, $U(A)^\dagger$, $V(B^T)$, $U(B^T)$) and performs two register swaps.

but play no role in the algorithm. Dually, the first row of $\mathcal{SP}(M)^\dagger$ contains the normalised entries of M; all remaining rows are irrelevant. Finally, in $\mathcal{MSP}(M)$, the rows of the matrix M are distributed among the columns with index a multiple of R. In particular, $\tilde{m}_{i,j} = \langle 0|^p \langle j|^r \mathcal{MSP}(M) |i\rangle^p |0\rangle^r$, with $i = 0, \ldots, P-1$ and $j = 0, \ldots, R-1$. All remaining entries of $\mathcal{MSP}(M)$ are irrelevant. We observe that while the standard state-preparation operator $\mathcal{SP}(M)$ encodes the row-wise vectorization of the matrix M into its first column, the modular state-preparation operator $\mathcal{MSP}(M)$ preserves the matrix's two-dimensional structure, encoding each row of M in a separate column.

3 The Two-Tower Algorithm

This section presents the Two-Tower algorithm in two stages. Section 3.1 recalls the two-matrix subroutine of [4], which serves as the foundation. Section 3.2 generalizes this construction to an arbitrary chain of K matrices.

3.1 Two-matrix multiplication

In [4], the authors introduce a quantum subroutine for the product of two matrices $A \in \mathbb{R}^{M\times N}$ and $B \in \mathbb{R}^{N\times S}$. Their original circuit, shown in Figure 2, uses operators $V(A)$, $U(A)^\dagger$, $V(B^T)$, $U(B^T)$ together with two register swaps on quantum registers $|0\rangle^n$, $|0\rangle^m$, and $|0\rangle^s$.

We observe that this circuit can be rearranged into the equivalent form shown in Figure 3, which serves as the base case ($K = 2$) of the Two-Tower construction. The key insight is

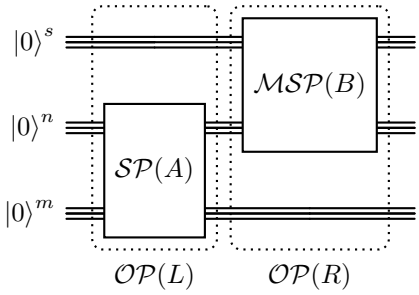


Figure 3 Circuit for computing $C = AB$, equivalent to Figure 2 but reorganised into the Two-Tower two-layer structure $\mathcal{OP}(L)$ and $\mathcal{OP}(R)$. Layer $\mathcal{OP}(L)$: $\mathcal{SP}(A) = U(A)V(A)$ on registers (n, m). Layer $\mathcal{OP}(R)$: $\mathcal{MSP}(B)$ on registers (s, n), a composite operator that applies the row-loading multiplexer $U(B)$ on (s, n) followed by $V(B)^\dagger$ on n to contract the shared index. This is the base case ($K = 2$) of the Two-Tower construction.

that the composition $U(A)^\dagger$ SWAP $U(B^T)V(B^T)$ SWAP $V(A)$ can be reorganised as the composition of $\mathcal{MSP}(B)$ and $\mathcal{SP}(A)$, eliminating the register swaps and grouping the operators into two sequential layers acting on disjoint register pairs. This reorganisation, while functionally identical for $K = 2$, reveals a two-layer structure — denoted by $\mathcal{OP}(L)$ and $\mathcal{OP}(R)$ — that generalises to arbitrary K. In particular, the rearranged circuit acts on $m + n + s$ qubits and produces:

$$(\mathbb{I}_M \otimes \mathcal{MSP}(B))\,(\mathcal{SP}(A) \otimes \mathbb{I}_S)\;|0\rangle^{m+n+s} = \frac{1}{\|A\|_F\,\|B\|_F} \sum_{i=0}^{M-1} \sum_{k=0}^{S-1} \underbrace{\sum_{j=0}^{N-1} a_{i,j}\, b_{j,k}}_{(AB)_{i,k}} |i\rangle^m\, |0\rangle^n\, |k\rangle^s + |\perp\rangle\,,$$

encoding the product AB in the outer registers with the internal index contracted to $|0\rangle^n$ (see Section 4.1 for a detailed derivation).

3.2 Chain multiplication for arbitrary K

The two-matrix construction of Section 3.1 generalises naturally to an arbitrary chain of K matrices $\mathcal{W} = M^{(0)}M^{(1)} \cdots M^{(K-1)}$. Before describing the full algorithm, we isolate the mechanism that makes the construction work.

3.2.1 The contraction mechanism

Recall from Definition 2 that $\mathcal{SP}(M)$ maps $|0, 0\rangle \mapsto \frac{1}{\|M\|_F} \sum_{i,j} m_{i,j}\,|i, j\rangle$. The adjoint $\mathcal{SP}(M)^\dagger$ reverses this map. Moreover, when $\mathcal{SP}(M)^\dagger$ acts on a basis state $|i, j\rangle$, it projects onto $|0, 0\rangle$ with amplitude proportional to the corresponding matrix entry $m_{i,j}$. Formally,

$$\mathcal{SP}(M)^\dagger\;|i,\,j\rangle \;=\; \frac{m_{i,j}}{\|M\|_F}\;|0,0\rangle \;+\; |\perp\rangle\,, \quad 0 \le i < P, \text{ and } 0 \le j < R, \tag{2}$$

where $|\perp\rangle$ is orthogonal to $|0, 0\rangle$. We observe that if two consecutive matrices $M^{(k)}$ and $M^{(k+1)}$ share a summation index encoded in the same quantum register, applying $\mathcal{SP}(M^{(k)})$ populates that register with amplitudes proportional to the entries of $M^{(k)}$, and $\mathcal{SP}(M^{(k+1)})^\dagger$ contracts it back to $|0\rangle$, multiplying each amplitude by the corresponding entry of $M^{(k+1)}$. By linearity, all such combinations occur simultaneously, and the resulting amplitudes reproduce exactly the classical matrix product. The Two-Tower circuit arranges all matrices so that every shared index is contracted through this mechanism.

3.2.2 Circuit structure

The circuit consists of two parallel layers (see Figure 4): $\mathcal{OP}(L)$ applies $\mathcal{SP}$ to every even-indexed matrix, and $\mathcal{OP}(R)$ applies $\mathcal{SP}^\dagger$ to every odd-indexed matrix, with one exception: when K is even, the last matrix $M^{(K-1)}$ has no paired right neighbour, and $\mathcal{MSP}(M^{(K-1)})$ is applied in place of $\mathcal{SP}(M^{(K-1)})^\dagger$ so that the output column register is preserved (see Definition 3 and Figure 4b). Since the operators within each layer act on non-overlapping registers, they execute in parallel, and the total circuit depth is independent of K. Following the intuition from the matrix-product vectorization discussed in the introduction, these two layers realise the Kronecker products of the first and last matrices of each internal subchain; Section 3.2.3 illustrates this in detail.

We observe that each matrix requires two registers to index rows and columns. However, registers are shared between adjacent matrices, so the overall circuit requires $K+1$ registers: two for the first matrix and one additional register for each remaining matrix. Thus, the circuit grows in qubit count while its depth remains constant — much like two towers rising floor by floor, which gives the algorithm its name. In Definition 4 we formalise the construction.

▶ **Definition 4** (Two-Tower operators). *Let $\mathcal{W} = M^{(0)}, \dots, M^{(K-1)}$ be a matrix chain with $K \geq 2$. Partition the chain into:*

- Left matrices *(even index):* $M^{(2i)} \in \mathbb{R}^{L_i \times R_i}$, *for* $0 \leq i \leq \lfloor (K-1)/2 \rfloor$;
- Right matrices *(odd index):* $M^{(2j+1)} \in \mathbb{R}^{R_j \times L_{j+1}}$, *for* $0 \leq j \leq \lfloor (K/2-1) \rfloor$.

Let us define $l_i = \log L_i$ and $r_i = \log R_i$ for the qubit counts. For each left matrix allocate a register pair $|0\rangle^{l_i} |0\rangle^{r_i}$; the initial state is

$$|\psi_0\rangle = \begin{cases} \bigotimes_{i=0}^{(K-1)/2} |0\rangle^{l_i} |0\rangle^{r_i}, & \text{if } K \text{ is odd} \\ \left(\bigotimes_{i=0}^{K/2-1} |0\rangle^{l_i} |0\rangle^{r_i}\right) |0\rangle^{l_{K/2}}, & \text{if } K \text{ is even.} \end{cases}$$

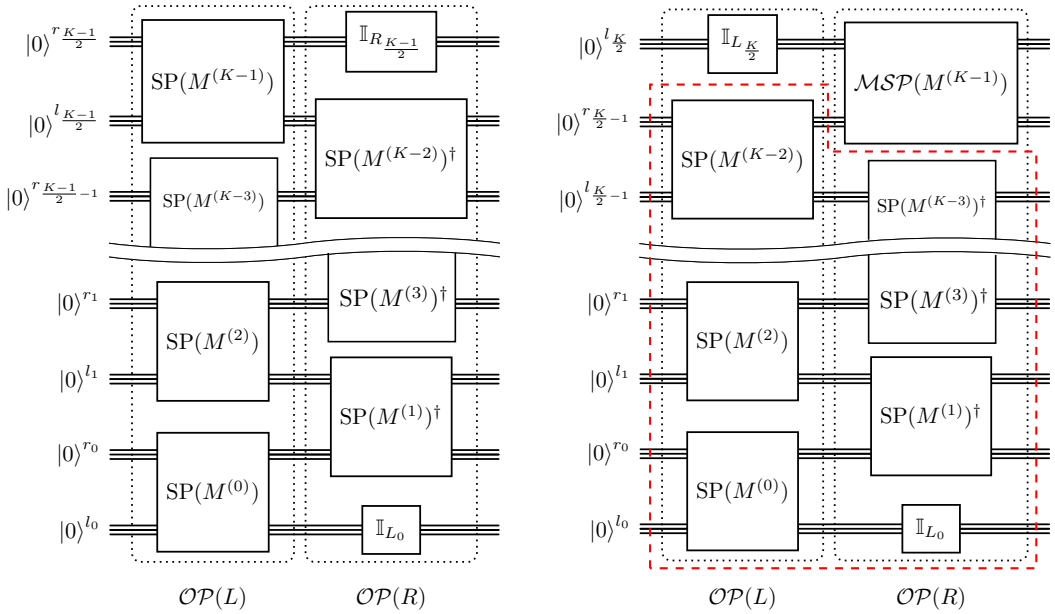
The circuit uses $\sum_{i=0}^{(K-1)/2}(l_i + r_i)$ qubits if K is odd and $\sum_{i=0}^{K/2-1}(l_i + r_i) + l_{K/2}$ if K is even. Define:

$$\mathcal{OP}(L) = \begin{cases} \bigotimes_{i=0}^{(K-1)/2} \mathcal{SP}\left(M^{(2i)}\right), & \text{if } K \text{ is odd} \\ \bigotimes_{i=0}^{K/2-1} \mathcal{SP}\left(M^{(2i)}\right) \otimes \mathbb{I}_{l_{\frac{K}{2}}}, & \text{if } K \text{ is even;} \end{cases} \tag{3}$$

$$\mathcal{OP}(R) = \begin{cases} \underbrace{\mathbb{I}_{L_0}}_{\text{left boundary}} \otimes \underbrace{\bigotimes_{j=0}^{(K-3)/2} \mathcal{SP}\left(M^{(2j+1)}\right)^\dagger}_{\text{contracts shared indexes}} \otimes \underbrace{\mathbb{I}_{R_{(K-1)/2}}}_{\text{right boundary}} & \text{if } K \text{ is odd} \\ \underbrace{\mathbb{I}_{L_0}}_{\text{left boundary}} \otimes \underbrace{\bigotimes_{j=0}^{K/2-2} \mathcal{SP}\left(M^{(2j+1)}\right)^\dagger}_{\text{contracts shared indexes}} \otimes \underbrace{\mathcal{MSP}(M^{(K-1)})}_{\text{right boundary}} & \text{if } K \text{ is even.} \end{cases} \tag{4}$$

The Two-Tower unitary *is $U_{\mathcal{W}} = \mathcal{OP}(R)\mathcal{OP}(L)$.*

In $\mathcal{OP}(L)$, each $\mathcal{SP}(M^{(2i)})$ acts on the register pair $(|0\rangle^{l_i}, |0\rangle^{r_i})$; all factors target disjoint registers and execute in parallel. In $\mathcal{OP}(R)$, each $\mathcal{SP}(M^{(2j+1)})^\dagger$ acts on the pair $(|\cdot\rangle^{r_j}, |\cdot\rangle^{l_{j+1}})$ — the boundary shared between adjacent even-matrix registers — and, by Equation (2), contracts it to $|0\rangle$, obtaining the corresponding entry of $M^{(2j+1)}$ as an amplitude up to a normalization factor. The identity operators $\mathbb{I}_{L_0}$ and $\mathbb{I}_{R_{(K-1)/2}}$ leave the outer indices l_0 and $r_{(K-1)/2}$ untouched; these encode the row and column indices of the product $\mathcal{W}$. The



(a) K odd. **(b)** K even.

Figure 4 Two-Tower circuit for a chain of K matrices. **(a)** K odd: registers are ordered top-to-bottom as $(r_{(K-1)/2}, l_{(K-1)/2}, \ldots, r_0, l_0)$. $\mathcal{OP}(L)$: all $\mathcal{SP}(M^{(2i)})$ act simultaneously on disjoint register pairs (r_i, l_i). $\mathcal{OP}(R)$: all $\mathcal{SP}(M^{(2j+1)})^\dagger$ act simultaneously on the interleaved pairs (l_{j+1}, r_j). **(b)** K even: an additional output register $l_{\frac{K}{2}}$ appears at the top. The last matrix $M^{(K-1)}$ uses the $\mathcal{MSP}$ state preparation (denoted $\mathcal{MSP}(M^{(K-1)})$), which acts on $(l_{\frac{K}{2}}, r_{K/2-1})$ within $\mathcal{OP}(R)$. This case can also be expressed as the product of two matrices: the result of applying the odd-case logic to the chain of the first $K-1$ matrices (dashed box in red), multiplied by the final matrix $M^{(K-1)}$ throught the $\mathcal{MSP}$ operator.

even-K case (see Figure 4b) requires $\mathcal{MSP}$ rather than $\mathcal{SP}^\dagger$ because $\mathcal{SP}^\dagger$ would contract both registers to $|0\rangle$, leaving no register to carry the output column index of $\mathcal{W}$; $\mathcal{MSP}$ contracts only the shared boundary register while keeping the output register free, yielding the same Frobenius-norm-scaled product.

Algorithm 1 summarises the complete procedure; open-source Qiskit and QCLAB implementations are publicly available[1].

3.2.3 Two-Tower Algorithm Intuition

The construction draws from a classical result in matrix theory. Lemma 4.3.1 of [17] expresses the product of three matrices via the Kronecker product and vectorization:

$$AXB = C \quad \Longleftrightarrow \quad (B^T \otimes A)\,\mathrm{vec}(X) = \mathrm{vec}(C) \tag{5}$$

In the quantum setting, preparing A and B on separate registers realises their Kronecker product naturally through the tensor-product structure of the Hilbert space. However, Equation (5) does not translate directly: $\mathcal{SP}(M)$ encodes any matrix in the statevector as a row-wise vectorization (Equation (1)), rather than as a matrix operator as Equation (5) requires for A and B. Consequently, whereas the classical formula $(B^T \otimes A)\,\mathrm{vec}(X)$ yields exactly $\mathrm{vec}(C)$, the Two-Tower circuit produces a higher-dimensional statevector containing

Algorithm 1 Two-Tower Matrix Multiplication

Require: Matrix chain $\mathcal{W} = (M^{(0)}, \ldots, M^{(K-1)})$ with $M^{(k)} \in \mathbb{R}^{P_k \times P_{k+1}}$, all dimensions powers of two.

Ensure: Quantum state encoding $\mathcal{W} = M^{(0)} \cdots M^{(K-1)}$ up to Frobenius-norm scaling.

1: Allocate register q_k of $\log P_k$ qubits for each $k = 0, \ldots, K-1$, and a final register q_K of $\log P_K$ qubits; initialise all to $|0\rangle$.
2: **Layer 1 (parallel).** For each even $k = 2i,\ 0 \le i \le \lfloor (K-1)/2 \rfloor$: apply $\mathcal{SP}(M^{(k)})$ on registers (q_{k+1}, q_k).
3: **Layer 2 (parallel).** For each odd $k = 2j+1,\ 0 \le j \le \lceil K/2 - 2 \rceil$: apply $\mathcal{SP}(M^{(k)})^\dagger$ on registers (q_{k+1}, q_k).
4: **if** K is even **then**
5: Apply $\mathcal{MSP}(M^{(K-1)})$ on (q_K, q_{K-1}).
6: **end if**
7: **return** The state $U_{\mathcal{W}} |0\rangle\ q_0$ holds the row index of $\mathcal{W}$, q_K holds the column index, and registers $q_1, \ldots, q_{K-1}$ are in state $|0\rangle$.

$\mathrm{vec}(C)$ alongside spurious contributions from index-mismatched combinations of entries of A, X, and B. A further difference is that the Two-Tower does not require any matrix transposition: each matrix is loaded directly through its own state-preparation operator, without reshaping B into B^T. The contraction mechanism (Equation (2)) is what separates the two: $\mathcal{SP}(X)^\dagger$ projects the valid, index-matched contributions onto the $|0\rangle$ subspace of the shared registers, while all spurious terms fall into the orthogonal complement $|\perp\rangle$. To illustrate this filtering and observe how the correct sums accumulate, we trace the circuit of Figure 5 for $D = ABC$ with $A \in \mathbb{R}^{M \times N}$, $B \in \mathbb{R}^{N \times S}$, $C \in \mathbb{R}^{S \times T}$. Let $m = \log M$, $n = \log N$, $s = \log S$, $t = \log T$, then we define the following quantum registers:

$$|\psi_0\rangle = |0\rangle^m |0\rangle^n |0\rangle^s |0\rangle^t .$$

Layer 1: apply $\mathcal{OP}(L) = \mathcal{SP}(A) \otimes \mathcal{SP}(C)$. Operator $\mathcal{SP}(A)$ acts on $(|0\rangle^m, |0\rangle^n)$ while $\mathcal{SP}(C)$ acts on $(|0\rangle^s, |0\rangle^t)$ simultaneously:

$$|\psi_1\rangle = \mathcal{OP}(L) |\psi_0\rangle = \frac{1}{\|A\|_F \|C\|_F} \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} \sum_{u=0}^{S-1} \sum_{v=0}^{T-1} a_{i,j}\, c_{u,v}\ |i\rangle^m |j\rangle^n |u\rangle^s |v\rangle^t .$$

Layer 2: apply $\mathcal{OP}(R) = \mathbb{I}_M \otimes \mathcal{SP}(B)^\dagger \otimes \mathbb{I}_T$. By Equation (2), $\mathcal{SP}(B)^\dagger$ acts on registers (n, s) picking up $\frac{b_{j,u}}{\|B\|_F}$ for each basis state $|j, u\rangle$. Applying to $|\psi_1\rangle$:

$$|\psi_2\rangle = \mathcal{OP}(R) |\psi_1\rangle = \frac{1}{\|A\|_F \|B\|_F \|C\|_F} \sum_{i=0}^{M-1} \sum_{v=0}^{T-1} \left(\sum_{j=0}^{N-1} \sum_{u=0}^{S-1} a_{i,j}\, b_{j,u}\, c_{u,v} \right) |i, 0, 0, v\rangle + |\perp\rangle .$$

The contraction of shared registers via Equation (2) is visible moving from $|\psi_1\rangle$ to $|\psi_2\rangle$: $\mathcal{SP}(B)^\dagger$ projects each basis state $|j\rangle^n |u\rangle^s$ onto $|0\rangle^n |0\rangle^s$ with amplitude $\frac{b_{j,u}}{\|B\|_F}$. The indices j and u are summed over in the classical product $(ABC)_{i,v} = \sum_{j,u} a_{i,j}\, b_{j,u}\, c_{u,v}$. In the Two-Tower circuit, these sums arise from the superposition over the shared registers (n, s): the adjoint state preparation $\mathcal{SP}(B)^\dagger$ converts each basis state $|j, u\rangle$ into the corresponding matrix entry, and the superposition accumulates all inner-product terms simultaneously. As a result, the double sum over (j, u) yields $d_{i,v} = (ABC)_{i,v}$, confirming that $|\psi_2\rangle$ encodes D in the outer registers (m, t), with the shared registers (n, s) contracted to $|0, 0\rangle$.

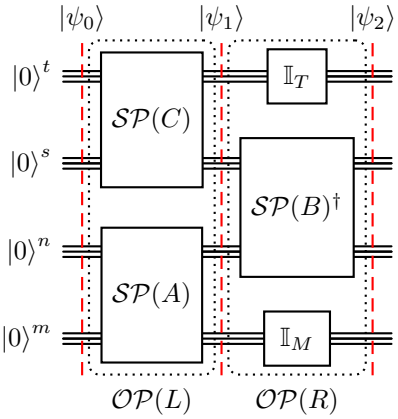


Figure 5 Two-Tower circuit for $D = ABC$ with $A \in \mathbb{R}^{M\times N}$, $B \in \mathbb{R}^{N\times S}$, $C \in \mathbb{R}^{S\times T}$, and $m = \log M$, $n = \log N$, $s = \log S$, $t = \log T$. $\mathcal{OP}(L)$**:** $\mathcal{SP}(C)$ on registers (t, s) and $\mathcal{SP}(A)$ on registers (n, m) execute in *parallel*. $\mathcal{OP}(R)$**:** $\mathcal{SP}(B)^\dagger$ on registers (s, n) contracts the shared summation index. The output state $|\psi_2\rangle$ encodes D in registers (t, m) (outer boundaries) with the internal registers (s, n) contracted to $|0\rangle$.

4 Analysis

We organize this section as follows: Section 4.1 proves that the Two-Tower circuit correctly encodes the chain product for any K. Section 4.2 derives the depth and qubit bounds. Section 4.3 discusses the normalisation factor and the signal weight.

4.1 Correctness

The correctness proof relies on the contraction mechanism of Equation (2), established in Section 3.2. The main result follows.

▶ **Theorem 5** (Two-Tower Matrix Multiplication)**.** *Let $\mathcal{W} = M^{(0)}, \ldots, M^{(K-1)}$ be a matrix chain with $M^{(k)} \in \mathbb{R}^{P_k\times P_{k+1}}$ for $k = 0, \ldots, K-1$, and with all $p_k = \log_2 P_k$. There exists a unitary $U_{\mathcal{W}} = \mathcal{OP}(R)\,\mathcal{OP}(L)$ such that*

$$\begin{aligned} U_{\mathcal{W}} :\ & |0\rangle^{p_0}\,|0\rangle^{p_1+\cdots+p_{K-1}}\,|0\rangle^{p_K} \\ \longmapsto\ & \frac{1}{\prod_{k=0}^{K-1}\left\|M^{(k)}\right\|_F}\sum_{p=0}^{P_0-1}\sum_{q=0}^{P_K-1} w_{p,q}\,|p\rangle^{p_0}\,|0\rangle^{p_1+\cdots+p_{K-1}}\,|q\rangle^{p_K} + |\perp\rangle\,, \end{aligned} \tag{6}$$

where $\mathcal{W} = M^{(0)}\cdots M^{(K-1)}$ and $w_{p,q} = (\mathcal{W})_{p,q}$. The term $|\perp\rangle$ is a superposition of basis states in which at least one intermediate (shared) register is not $|0\rangle$; it is orthogonal to the subspace $\mathcal{H}_{\mathcal{W}} = \mathrm{span}\{|p, 0, \ldots, 0, q\rangle\}$ for $0 \le p < P_0$, $0 \le q < P_K$ The circuit $U_{\mathcal{W}}$ has:

- ***Depth*** $\mathcal{O}\big(\max_{k=0}^{K-1}\ \mathrm{polylog}(P_k P_{k+1})\big)$*, independent of K;*
- ***Qubits*** $O\big(\sum_{k=0}^{K}\log P_k\big)$.

Proof of Theorem 5 for odd K. Using the notation of Section 3.2: left matrices $M^{(2i)} \in \mathbb{R}^{P_k\times P_{k+1}}$ (where $k = 2i$) and right matrices $M^{(2j+1)} \in \mathbb{R}^{P_{k+1}\times P_{k+2}}$ (where $k = 2j$), with shared registers shared between adjacent pairs.

Step 1. Since all factors of $\mathcal{OP}(L) = \bigotimes_i \mathcal{SP}(M^{(2i)})$ act on disjoint registers:

$$|\psi_1\rangle = \frac{1}{\prod_i \left\|M^{(2i)}\right\|_F} \sum_{\vec{l},\vec{r}} \left(\prod_i m^{(2i)}_{l_i,r_i}\right) |l_0\rangle \left|r_0,\, l_1,\, r_1, \ldots, r_{(K-1)/2-1}\, l_{(K-1)/2}\right\rangle \left|r_{(K-1)/2}\right\rangle .$$

Step 2. By Equation (2), each $\mathcal{SP}(M^{(2j+1)})^\dagger$ on shared pair $|r_j, l_{j+1}\rangle$ contributes amplitude $\frac{m^{(2j+1)}_{r_j,l_{j+1}}}{\left\|M^{(2j+1)}\right\|_F}$ on $|0,0\rangle$. Applying all $(K-1)/2$ such operators in parallel and collecting:

$$\mathcal{OP}(R)\,|\psi_1\rangle = \frac{1}{\prod_k \left\|M^{(k)}\right\|_F} \sum_{\vec{l},\vec{r}} \left(\prod_i m^{(2i)}_{l_i,r_i}\right)\left(\prod_j m^{(2j+1)}_{r_j,l_{j+1}}\right) \left| l_0, \underbrace{0,\ldots,0}_{\text{shared}}, r_{(K-1)/2} \right\rangle + |\perp\rangle .$$

Observing that

$$\sum_{\vec{l},\vec{r}} \left(\prod_i m^{(2i)}_{l_i,r_i}\right)\left(\prod_j m^{(2j+1)}_{r_j,l_{j+1}}\right) = \sum_{r_0,l_1,\ldots,l_{(K-1)/2}} m^{(0)}_{l_0,r_0} \cdots m^{(K-1)}_{l_{(K-1)/2},r_{(K-1)/2}} = w_{l_0,\, r_{(K-1)/2}} ,$$

and setting $p \leftarrow l_0$, $q \leftarrow r_{(K-1)/2}$ yields Equation (6). ◀

When K is even, the chain contains $K/2$ even-indexed matrices $M^{(0)}, M^{(2)}, \ldots, M^{(K-2)}$ and $K/2$ odd-indexed matrices $M^{(1)}, M^{(3)}, \ldots, M^{(K-1)}$. The last odd-indexed matrix $M^{(K-1)}$ has no paired right neighbour, so a plain $\mathcal{SP}(M^{(K-1)})^\dagger$ would contract both registers to $|0\rangle$, leaving no register to carry the output column index. Instead, we encode $M^{(K-1)}$ via the modular state-preparation operator $\mathcal{MSP}(M^{(K-1)})$.

Proof of Theorem 5 for even K. Apply the odd-K argument of Section 4.1 to the first $K-1$ matrices (an odd-length chain). Then, we append a fresh ancilla register of $p_K = \log_2 P_K$ qubits for the final column index. The intermediate state after processing the first $K-1$ matrices of the chain $Z = M^{(0)} \cdots M^{(K-2)}$ is:

$$|\psi_1\rangle = \frac{1}{\prod_{k=0}^{K-2} \left\|M^{(k)}\right\|_F} \sum_{l_0=0}^{P_0-1} \sum_{d=0}^{P_{K-1}-1} z_{l_0,d} \left| l_0, \underbrace{0,\ldots,0}_{\text{shared}}, d, \underbrace{0}_{\text{ancilla}} \right\rangle \;+\; |\perp\rangle ,$$

where $z_{l_0,d} = (M^{(0)} \cdots M^{(K-2)})_{l_0,d}$ and the ancilla register remains in state $|0\rangle$. Projecting $|\psi_1\rangle$ onto the outer registers (p_0, p_{K-1}) yields exactly $\mathcal{SP}(Z)$. We then complete the chain product by applying the modular state-preparation operator $\mathcal{MSP}(M^{(K-1)})$ as in the two-matrix circuit of Section 3.1, since $\mathcal{W} = ZM^{(K-1)}$. ◀

4.2 Complexity

The depth and qubit bounds stated in Theorem 5 derive from the structure of the two-layer circuit and the state-preparation cost. In particular:

- **Depth.** Each $\mathcal{SP}(M^{(k)})$ and $\mathcal{SP}(M^{(k)})^\dagger$ has depth $\mathcal{O}(\mathrm{polylog}(P_k P_{k+1}))$. Since all even-indexed $\mathcal{SP}$ operators execute in parallel on the first layer, and all odd-indexed $\mathcal{SP}^\dagger$ operators execute in parallel on the second layer, the total depth is $\mathcal{O}(\max_k \mathrm{polylog}(P_k P_{k+1}))$, independent of K.
- **Qubits.** The circuit allocates one register of $p_k = \log_2 P_k$ qubits for each matrix $M^{(k)}$, $k = 0, \ldots, K-1$, and a final register $p_K = \log_2 P_K$ qubits for the output column index. Adjacent matrices share a shared register already counted in the allocation for $M^{(k+1)}$, and no additional ancillae are required. The total is $\sum_{k=0}^{K} \log_2 P_k = \Theta\big(\sum_{k=0}^{K} \log_2 P_k\big)$.

4.3 Normalisation and signal weight

The amplitudes in Equation (6) carry the *normalisation factor* $\mathcal{N} = \frac{1}{\prod_{k=0}^{K-1} \|M^{(k)}\|_F}$, which we compute classically before running the circuit. The residual term $|\perp\rangle$ lies in the orthogonal complement of $\mathcal{H}_\mathcal{W} = \text{span}\{|p, 0, \ldots, 0, q\rangle\}$ for $0 \le p < P_0$ and $0 \le q < P_K$ and has squared norm $1 - \|\mathcal{W}\|_F^2 \mathcal{N}^2$; this term does not indicate a failure of the subroutine but is the component of the unitary evolution that falls outside the subspace encoding the product. We therefore define the *signal weight* as the total squared amplitude carried by the basis states in $\mathcal{H}_\mathcal{W}$ — those whose amplitudes, up to the normalisation factor $\mathcal{N}$, are exactly the entries of the chain product:

$$\Omega_\mathcal{W} = \frac{\|\mathcal{W}\|_F^2}{\prod_{k=0}^{K-1} \left\|M^{(k)}\right\|_F^2}.$$

In a standalone algorithm, $\Omega_\mathcal{W}$ would correspond to the success probability of post-selecting on $\mathcal{H}_\mathcal{W}$. However, the Two-Tower — much like the QFT — is designed as a subroutine with a downstream algorithm applied before any measurement; we therefore use the term *signal weight* to emphasise that $\Omega_\mathcal{W}$ is a parameter inherited by the subsequent computation, not a measurement outcome of the circuit itself.

For example a special case where $\Omega_\mathcal{W}$ is absorbed analytically is the estimation of $\|\mathcal{W}\|_F$: since $\Omega_\mathcal{W} = \|\mathcal{W}\|_F^2 / \prod_k \|M^{(k)}\|_F^2$ and the denominator is classically known, Amplitude Estimation directly yields $\|\mathcal{W}\|_F$ without any penalty from the signal weight.

By sub-multiplicativity of the Frobenius norm ($\|AB\|_F \le \|A\|_F \|B\|_F$), $\Omega_\mathcal{W} \le 1$, with $\Omega_\mathcal{W} = 1$ if and only if $\|\mathcal{W}\|_F = \prod_{k=0}^{K-1} \|M^{(k)}\|_F$, and this happens in special cases. For example, consider K copies of a normal $N \times N$ matrix A, i.e., a matrix such that $A^\dagger A = AA^\dagger$ and denote its singular values by $\sigma_1 \ge \cdots \ge \sigma_N \ge 0$. Since $\|A\|_F^2 = \sum_i \sigma_i^2$ and singular values of $\mathcal{W} = A^K$ when A is normal are σ_i^K, we get $\left\|A^K\right\|_F^2 = \sum_i \sigma_i^{2K}$. The signal weight becomes

$$\Omega_\mathcal{W} = \frac{\left\|A^K\right\|_F^2}{\|A\|_F^{2K}} = \frac{\sum_i \sigma_i^{2K}}{(\sum_i \sigma_i^2)^K},$$

which for large K is dominated by the largest singular value: $\Omega_\mathcal{W} \approx (\sigma_1^2 / \|A\|_F^2)^{K-1}$. In general, $\Omega_\mathcal{W}$ can decrease with K; the rate of decay depends on the spectral structure of the matrices. Two edge cases are instructive.

- *Well-conditioned matrices* ($\sigma_1/\sigma_N \approx 1$): all singular values are approximately equal to the same σ, so $\|A^K\|_F^2 / \|A\|_F^2 \approx N\sigma^{2K}/(N\sigma^2)^K$, giving $\Omega_\mathcal{W} \approx N^{-(K-1)}$. The signal weight decays exponentially with K: the information spreads across many directions and repeated multiplication reduces the overall signal strength. Note that the unitary case ($\sigma = 1$) belongs to this category, since $\Omega_\mathcal{W} = N/N^K = 1/N^{K-1}$.
- *Near-rank-one matrices* ($\sigma_2 \approx 0$): a single singular value dominates the Frobenius norm, so $\Omega_\mathcal{W} \approx \frac{\sigma_1^{2K}}{\|A\|_F^{2K}} \approx 1$. The matrix acts essentially as a scalar along one direction, and repeated multiplications do not weaken the signal.

The signal weight is therefore data-dependent: matrices with flat spectra cause rapid decay, whereas matrices with peaked spectra maintain $\Omega_\mathcal{W}$ close to unity. This phenomenon is counterintuitive: in most numerical settings, well-conditioned matrices are the favorable case, but here a flat spectrum spreads the signal uniformly across all singular directions, diluting it exponentially with K. Nevertheless, we observe that Amplitude Amplification [8] can boost $\Omega_\mathcal{W}$ to any desired constant, at the cost of $\mathcal{O}(1/\sqrt{\Omega_\mathcal{W}})$ calls to the Two-Tower circuit and its inverse, introducing a dependence on K.

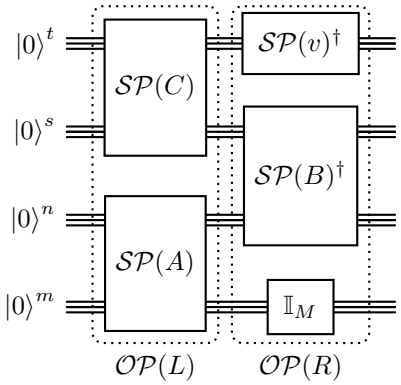


Figure 6 Chain-vector circuit for $w = ABCv$ with $A \in \mathbb{R}^{M\times N}$, $B \in \mathbb{R}^{N\times S}$, $C \in \mathbb{R}^{S\times T}$, $v \in \mathbb{R}^{T\times 1}$. Since $P_K = 1$, the output register vanishes and the $\mathcal{MSP}$ decomposition of the last factor reduces to $\mathcal{SP}(v)^\dagger$ on the t-qubit boundary register alone. The product $w = ABCv \in \mathbb{R}^M$ is encoded in the m-qubit register, with all shared registers (t, s, n) contracted to $|0\rangle$.

5 Matrix-Chain-Vector Multiplication

An important special case of Theorem 5 arises when the last matrix in the chain is a column vector. This case finds direct application in iterative solvers, the power method, and Krylov subspace methods [15], all of which require repeated matrix-vector products. The following corollary formalises this specialisation.

▶ **Corollary 6** (Matrix-chain-vector multiplication). *Setting $P_K = 1$ in Theorem 5 identifies $M^{(K-1)}$ with a column vector $v \in \mathbb{R}^{P_{K-1}}$. The right-boundary register vanishes ($\log 1 = 0$ qubits), and the Two-Tower encodes the product $w = M^{(0)} \cdots M^{(K-2)} v \in \mathbb{R}^{P_0}$ as the amplitudes of the $\log P_0$-qubit output register, in depth $\mathcal{O}(\max_k \operatorname{polylog}(P_k P_{k+1}))$ using $\Theta\big(\sum_{k=0}^{K-1} \log P_k\big)$ qubits.*

Proof. Immediate from Theorem 5 with $P_K = 1$: the $\log_2 P_K = 0$ qubits vanish from both the register allocation and the output state, and all remaining bounds carry over unchanged. ◀

We validate this specialisation at machine precision using open-source Qiskit and QCLAB implementations[1], confirming that the circuit correctly produces the chain-vector product for all tested instances. Figure 6 illustrates the case $K = 4$ with $w = ABCv$: the column-index register is no longer needed and $\mathcal{MSP}(v)$ reduces to $\mathcal{SP}(v)^\dagger$ on the boundary register alone, so the product vector is encoded entirely in the bottom register.

6 Conclusions and Future Work

We presented the Two-Tower Matrix Multiplication, a quantum subroutine that encodes $\mathcal{W} = M^{(0)} \cdots M^{(K-1)}$, $M^{(k)} \in \mathbb{R}^{P_k \times P_{k+1}}$, using $\Theta\big(\sum_{k=0}^{K} \log P_k\big)$ qubits and circuit depth $\mathcal{O}(\max_k \operatorname{polylog}(P_k P_{k+1}))$, independent of K. The construction generalises the two-matrix operators of [4] to arbitrary chain length by interleaving $\mathcal{SP}$ and $\mathcal{SP}^\dagger$ gates on disjoint registers; the $\mathcal{SP}^\dagger$ contraction drives all shared registers to $|0\rangle$, producing the chain product as amplitudes. Compared with classical algorithms, the Two-Tower trades sequential $\mathcal{O}(K \cdot N^\omega)$ depth — or $\mathcal{O}(\log K \cdot \log N)$ with unbounded parallelism — for $\mathcal{O}(\operatorname{polylog}(N))$ quantum depth. The advantage materialises when the downstream computation requires only aggregate

quantities of $\mathcal{W}$ extractable via Amplitude Estimation. Representative instances include: norm, trace, and variance estimation via Amplitude Estimation [2, 3, 9, 26]; graph-matrix powers A^K for spectral analysis, walk counting, and mixing estimation [24, 18]; linear systems with chain-structured operators via HHL [16] or Neumann-series LCU [10]; and kernel evaluation in quantum machine learning [6, 21]. The chain-vector specialisation opens connections to iterative numerical methods — including the power method ($A^t v$), Krylov subspace construction ($\{v, Av, \ldots, A^{t-1}v\}$), iterative refinement via Neumann series, and polynomial matrix evaluation $p(A)v$ — where the Two-Tower provides a depth-efficient building block for each matrix-vector product in the iteration.

Several directions for future work emerge from the present construction.

- *Trading depth for width.* As shown in Section 4.3, the signal weight $\Omega_{\mathcal{W}}$ can decrease with the chain length K. A natural mitigation is a *hybrid strategy* that alternates between the Two-Tower and block-encoding: the Two-Tower computes partial chain products in constant depth at the cost of growing qubit count and decaying signal weight, while block-encoding stages absorb the intermediate results and continue the multiplication in depth rather than width. This approach trades qubit overhead for circuit depth within the same computation, but it requires an efficient conversion between statevector encoding and block encoding — an open problem that we leave for future investigation.
- *Complex matrices.* The current construction assumes real entries. Extending to complex-valued matrices requires loading the entry-wise conjugate $\overline{M^{(k)}}$ in place of $M^{(k)}$ within $\mathcal{OP}(R)$, so that the contraction mechanism produces the correct Hermitian inner products; the rest of the circuit remains unchanged. A formal proof and numerical validation are left for future work.
- *Unitary chain encoding.* An interesting generalisation replaces the matrices $M^{(0)}, M^{(1)}, \ldots, M^{(K-1)}$ with arbitrary unitary operators $U_0, \ldots, U_{K-1}$, encoding their product as a quantum state. Each U_k would enter the circuit through a state-preparation oracle that places the vectorized unitary in its first column. If the contraction mechanism extends to this setting, the Two-Tower could compute transition amplitudes $\langle\phi| U_0 \cdots U_{K-1} |v\rangle$ at depth independent of K, with direct applications in quantum simulation and process tomography. We leave the formal analysis of this direction for future work.

References

1. J. Alman and V. V. Williams. A refined laser method and faster matrix multiplication. In *Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 522–539. SIAM, 2021.
2. G. Antonioli, A. Berti, A. Poggiali, A. Bernasconi, and G. M. Del Corso. Outlier detection and other applications of quantum matrix multiplication. In *2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 509–518. IEEE, 2025.
3. A. Bernasconi, A. Berti, G. M. Del Corso, R. Guidotti, and A. Poggiali. Quantum subroutine for variance estimation: algorithmic design and applications. *Quantum Machine Intelligence*, 6(2):78, 2024.
4. A. Bernasconi, A. Berti, G. M. Del Corso, and A. Poggiali. Quantum subroutine for efficient matrix multiplication. *IEEE Access*, 12:116274–116284, 2024. doi:10.1109/ACCESS.2024.3446176.
5. A. Berti and F. Ghisoni. Efficient quantum state preparation with bucket brigade QRAM. *arXiv preprint arXiv:2510.16149*, 2025.
6. J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd. Quantum machine learning. *Nature*, 549(7671):195–202, 2017.

7 D. Bini, M. Capovani, F. Romani, and G. Lotti. $o(n^2.7799)$ complexity for $n \times n$ approximate matrix multiplication. *Information Processing Letters*, 8:234–235, 1979. doi:10.1016/0020-0190(79)90113-3.

8 G. Brassard, P. Høyer, M. Mosca, and A. Tapp. Quantum amplitude amplification and estimation. *Contemporary Mathematics*, 305:53–74, 2002. arXiv:quant-ph/0005055.

9 C. Cade and A. Montanaro. The quantum complexity of computing Schatten p-norms. *arXiv preprint arXiv:1706.09279*, 2017.

10 S. Chakraborty, A. Gilyén, and S. Jeffery. The Power of Block-Encoded Matrix Powers: Improved Regression Techniques via Faster Hamiltonian Simulation. In *46th International Colloquium on Automata, Languages, and Programming (ICALP 2019)*, volume 132 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 33:1–33:14. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019. doi:10.4230/LIPIcs.ICALP.2019.33.

11 R. Duan, H. Wu, and R. Zhou. Faster matrix multiplication via asymmetric hashing. In *Proceedings of the 64th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 2129–2138. IEEE, 2023.

12 D. Fang, L. Lin, and Y. Tong. Time-marching based quantum solvers for time-dependent linear differential equations. *Quantum*, 7:955, March 2023. URL: http://dx.doi.org/10.22331/q-2023-03-20-955, doi:10.22331/q-2023-03-20-955.

13 A. Gilyén, Y. Su, G. H. Low, and N. Wiebe. Quantum singular value transformation and beyond: Exponential improvements for quantum matrix arithmetics. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing (STOC 2019)*, pages 193–204. ACM, 2019. doi:10.1145/3313276.3316366.

14 V. Giovannetti, S. Lloyd, and L. Maccone. Quantum random access memory. *Physical Review Letters*, 100(16):160501, 2008.

15 G. H. Golub and C. F. Van Loan. *Matrix computations*. Johns Hopkins University Press, Baltimore, MD, third edition, 1996.

16 A. W. Harrow, A. Hassidim, and S. Lloyd. Quantum algorithm for linear systems of equations. *Physical Review Letters*, 103(15):150502, 2009.

17 R. A Horn and C. R. Johnson. *Matrix analysis*. Cambridge university press, 2012.

18 D. Janzing and P. Wocjan. BQP-complete problems concerning mixing properties of classical random walks on sparse graphs. *arXiv preprint quant-ph/0610235*, 2006.

19 A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta. Quantum computing with Qiskit, 2024. arXiv:2405.08810, doi:10.48550/arXiv.2405.08810.

20 S. Keip, D. Camps, and R. Van Beeumen. QCLAB: A matlab toolbox for quantum computing. In *2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 1175–1181. IEEE, 2025.

21 I. Kerenidis and A. Prakash. Quantum Recommendation Systems. In *8th Innovations in Theoretical Computer Science Conference (ITCS 2017)*, volume 67 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 49:1–49:21. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2017. doi:10.4230/LIPIcs.ITCS.2017.49.

22 X. Li, P.-L. Zheng, C. Pan, F. Wang, C. Cui, and X. Lu. Faster quantum subroutine for matrix chain multiplication via Chebyshev approximation. *Scientific Reports*, 15(1):28559, 2025.

23 A. Montanaro and C. Shao. Quantum and classical query complexities of functions of matrices. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*, pages 573–584, 2024.

24 N. A. Nghiem and T.-C. Wei. Quantum algorithm for estimating largest eigenvalues. *Physics Letters A*, 488:129138, 2023.

25 M. A. Nielsen and I. L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.

26 A. Poggiali, A. Bernasconi, A. Berti, G. M. Del Corso, R. Guidotti, et al. Quantum feature selection with variance estimation. In *ESANN*, 2023.

27 A. Poggiali and J. Ju. A more efficient quantum circuit for estimating the variance. *Quantum Machine Intelligence*, 8(1):34, 2026.

28 L. Wossnig, Z. Zhao, and A. Prakash. Quantum linear system algorithm for dense matrices. *Physical Review Letters*, 120(5):050502, 2018. doi:10.1103/PhysRevLett.120.050502.